



CLAUDE CODE · GETTING STARTED

Claude Code 시작 가이드

설치부터 간단한 활용 방법까지 —
따라하면 되는 Claude Code 입문

```
PowerShell — ccfm\report-pipeline

PS C:\Users\ccfm> cClaude

* Welcome to Claude Code · v2.1

> 광고 리포트 CSV 3개를 공통 스키마로 합치고 .xlsx 로 저장해줘

• 프로젝트 탐색 중...
  □ Read data/*.csv (3 files · 12,480 rows)
  □ Write scripts/merge.py
  □ Bash python scripts/merge.py
✓ weekly_summary.xlsx 생성 완료 (4 sheets)

> 다음 주부터는 자동으로 돌리게
```

목차 · Table of Contents

설치부터 활용까지 — 8개 챕터, 약 20분 소요

01 기능 요약 (What & Why) 3분

02 기존 방식과의 비교 2분

03 핵심 이점 & 변화 2분

04 Windows 설치 가이드 5분

05 첫 실행 & 기본 명령 3분

06 장단점 정리 2분

07 CCFM 실무 활용 사례 & 주의사항 3분

08 템플릿 사용 가이드 (내부용) —

Claude Code란 무엇인가?

터미널에서 직접 돌아가는 AI 코딩 에이전트 — 채팅창이 아니라 당신의 작업 환경 안으로 들어옵니다.

01

터미널 네이티브 (Terminal-native)

브라우저·IDE 플러그인이 아닌 CLI에서 실행. 당신의 파일, 커밋, 환경변수, 툴체인에 직접 접근합니다.

02

맥락 이해 (Context-aware)

현재 디렉토리의 전체 코드베이스를 읽고 수정합니다. 파일 하나씩 복붙할 필요가 없어요.

03

에이전틱 실행 (Agentic)

명령 대신 목표를 줍니다. Claude가 계획을 세우고, 파일을 고치고, 테스트까지 돌려보냅니다.

무엇이 달라지는가

챗봇에 붙여넣던 방식과 비교해, 실제 작업 흐름이 어떻게 바뀌는지.

BEFORE · 기존 방식

복붙 반복 워크플로우

- 코드를 브라우저 챗창에 복사
- 답변을 다시 IDE로 붙여넣기
- 파일 여러 개면 한 번에 처리 불가
- 실행·검증은 전부 수동
- 맥락이 끊겨 대화가 길어질수록 품질 저하



AFTER · CLAUDE CODE

터미널에서 완결되는 흐름

- 프로젝트 루트에서
- ```
claude
```
- Claude가 파일 탐색·수정·실행까지
  - 여러 파일 동시 편집 가능
  - 터미널 명령·테스트 자동 수행
  - 세션 내 프로젝트 맥락 유지

## 쓰면서 체감하게 되는 세 가지 변화

**3-5x**

더 빠른 작업 속도

### 반복 업무의 자동화

로그 파싱, CSV 정제, 보일러플레이트 코드  
— “한 번 말하면” 끝납니다. AX팀 내부 측정 기준.

**0 →**

컨텍스트 스위칭

### IDE를 떠날 필요 없음

브라우저·챗창을 오갈 필요가 없습니다. 실무자의 주의집중 유지가 가장 큰 효과입니다.

**+1**

팀 역량

### 실무자가 직접 자동화

AX팀에 매번 요청 대신, 본인 도메인 지식 기반으로 직접 스크립트·파이프라인을 만듭니다.

# Windows 설치 — 네 단계

Windows는 WSL2(Ubuntu) 위에서 실행하는 것이 공식 권장 방식입니다.

## STEP 01

### WSL2 설치

PowerShell을 관리자 권한으로 열고  
실행. 재부팅 필요.

```
wsl --install
```

## STEP 02

### Ubuntu 계정 생성

재부팅 후 터미널에 Ubuntu 창이 뜹니  
다. 사용자명 · 비밀번호 설정.

```
username / password
```

## STEP 03

### Node.js 설치

WSL(Ubuntu) 터미널에서  
NodeSource LTS를 설치합니다.

```
sudo apt install -y nodejs
npm
```

## STEP 04

### Claude Code 설치

npm 글로벌 설치 후 첫 실행 시 로그인  
안내가 뜹니다.

```
npm install -g @anthropic-a
i/claude-code
```

스크린샷 슬롯 · 설치 완료 화면

# 첫 실행 — 프로젝트 폴더에서 `claude`

# 1. 작업할 프로젝트 폴더로 이동

```
$ cd ~/projects/my-report
```

# 2. Claude Code 실행

```
$ claude
```

\* Welcome to Claude Code

/help 로 명령 목록 확인

```
> data/raw.csv 에서 중복 행을 제거하고
cleaned.csv 로 저장해줘
```

- 파일을 읽는 중... (1,204 rows)
- pandas 스크립트 생성 중...
- ✓ cleaned.csv 생성 완료 (1,087 rows)

## 자주 쓰는 명령어

**/init**

프로젝트 맥락 파일(CLAUDE.md) 생성

**/clear**

대화 맥락 초기화 (긴 세션 후)

**Ctrl + C**

실행 중인 작업 중단

# 장단점 — 솔직하게

모든 업무가 Claude Code에 적합한 건 아닙니다. 맞는 일에 쓰세요.

## ✓ Pros · 장점

- 로컬 파일/커맨드 직접 접근 — 복붙 없음
- 반복 작업(데이터 정제, 리팩토링) 압도적 속도
- 프로젝트 맥락 기반 응답 — 품질 일관성
- 비개발자도 자연어로 스크립트 생성 가능
- Git, 테스트, 터미널 통합 — 실무 적합도 높음

## ✗ Cons · 단점

- WSL 등 초기 환경 세팅 허들 (Windows)
- 대형 코드베이스에서 토큰 비용 증가
- 자동 실행 권한 — 승인 전 반드시 검토 필요
- 실시간 UI/디자인 피드백에는 부적합
- 사내 데이터 반출 관련 내부 정책 확인 필수

# CCFM 실무 활용 & 주의사항

## 활용 사례

### MEDIA · 리포트

#### 광고 리포트 CSV 자동 정제 & 대시보드 포맷 변환

매체사별 원본 파일 → 공통 스키마 → PPT용 표 자동 생성까지 한 세션.

### CREATIVE · 텍스트

#### 기획안 초고 → 검수용 비교 버전 3종 생성

톤·길이·타깃별 버전을 폴더에 파일로 저장. 리뷰 워크플로우 단축.

### OPS · 문서

#### 회의록 폴더 일괄 요약 & Notion 이관 스크립트

transcript 폴더 전체 → 요약 md → Notion API 업로드 자동화.

## 주의사항

### DO

작업 전 **Git 커밋**을 습관화하세요. 되돌릴 지점이 반드시 필요합니다.

### DO

민감 데이터가 포함된 파일은 **로컬 샘플본**으로 먼저 테스트하세요.

### DON'T

자동 실행(--dangerously-skip-permissions) 플래그는 **검증된 스크립트에만**.

### DON'T

고객사 원본 데이터·자격증명을 그대로 컨텍스트에 넣지 마세요. **사내 정책을 먼저 확인**.

# 템플릿 사용 가이드

콘텐츠를 바꿔도 레이아웃이 깨지지 않는 네 가지 원칙.

## 1 슬롯(data-slot) 표기를 따르세요

각 콘텐츠 영역에는 `data-slot="title · max 2 lines"` 같은 제약이 붙어 있습니다. Tweaks에서 **Show slots**를 켜면 슬롯이 시각화됩니다.

## 2 텍스트 길이 한도를 지키세요

- Title: 최대 2줄 (약 30자)
- Lede: 최대 3줄 (약 80자)
- 3-up 카드 설명문: 최대 3줄
- Do/Don't 각 항목: 1문장 원칙

## 3 레이아웃을 새로 만들지 마세요

슬라이드 유형은 **Cover · TOC · 3-up · Before/After · Steps · Code · Pros/Cons · UseCase · Divider**로 고정. 새 주제는 기존 유형 중에서 고르세요.

## 4 Tweaks로 톤을 조정하세요

- 액센트 색 — 주제별로 구분 (Claude=Rausch)
- Dark/Light — 장표 성격에 따라
- Font scale — 현장 화면 크기 맞춤
- Footer on/off — 인쇄 vs. 스크린